

Chap3-2 HW: Matrix Multiplication Optimization

112006265

蔡定宇

EECS (GS)

`matmul_blocked()` is faster because it reduces cache misses

10 HOTTEST FUNCTIONS (Sort Event - CYCLES_NOT_IN_HALT)

FUNCTION	%L1_DC_MISSES
<code>matmul(float*, float*, float*, int, int, int)</code>	29.6302
<code>matmul_blocked(float*, float*, float*, int, int, int)</code>	0.8795
<code>matmul_avx2_blocked(float*, float*, float*, int, int, int)</code>	4.9796
<code>matmul_avx2(float*, float*, float*, int, int, int)</code>	14.8499
<code>asm_sysvec_apic_timer_interrupt</code>	8.4545
<code>native_write_msr</code>	0
<code>perf_adjust_freq_unthr_context</code>	0
<code>clear_page_erms</code>	50
<code>scheduler_tick</code>	0
<code>copy_page_from_iter_atomic</code>	0

`matmul_avx2_blocked()` is faster because it keeps the cache-friendly blocked structure and additionally uses AVX2 SIMD to compute multiple elements per instruction.

10 HOTTEST FUNCTIONS (Sort Event - CYCLES_NOT_IN_HALT)

FUNCTION	RETIRED_INST	IPC	CPI
<code>matmul(float*, float*, float*, int, int, int)</code>	302526	0.2893	3.4569
<code>matmul_blocked(float*, float*, float*, int, int, int)</code>	126378	2.9296	0.3413
<code>matmul_avx2(float*, float*, float*, int, int, int)</code>	28414	0.9631	1.0384
<code>asm_sysvec_apic_timer_interrupt</code>	5193	0.3088	3.238
<code>matmul_avx2_blocked(float*, float*, float*, int, int, int)</code>	33926	2.6412	0.3786
<code>native_write_msr</code>	74	0.1655	6.0405
<code>pvclock_clocksource_read_nowd</code>	295	0.8912	1.122
<code>update_load_avg</code>	359	1.1149	0.8969
<code>update_curr</code>	143	0.4719	2.1189
<code>sync_regs</code>	11	0.0425	23.5455